EPFL - Computer Science

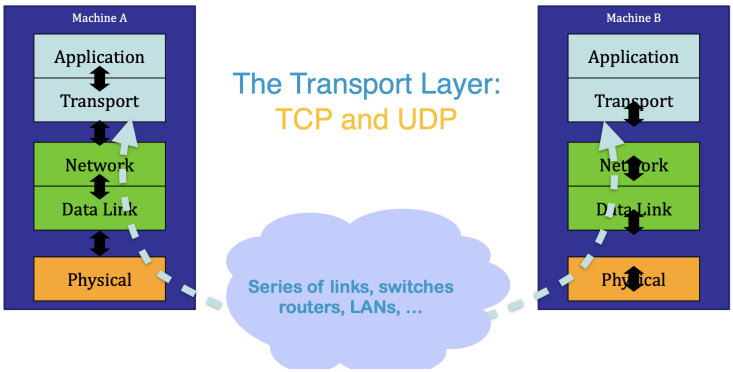
COM-402 Advanced Networks

Lecture 07 — January 18, 2026

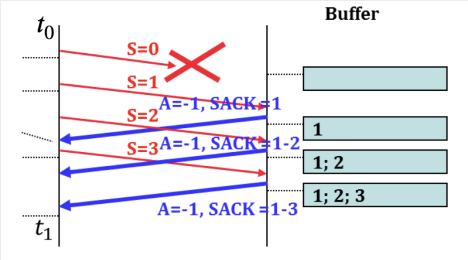
Author: Jonathan Dutoit

Professor: Pavlos Nikoupoulos

Transporter Layer Continued



Window size = 4'000 bytes, one packet = 1'000 bytes. At time t_0, the usable part of the sliding window is 4000, and there is a lot of data in the socket to be sent. So, the sender sends 4 segments. At time t_1, sender...



A. Can send segment S=4
B. Cannot send segment S=4
C. It depends on whether data was consumed by application
D. I do not know

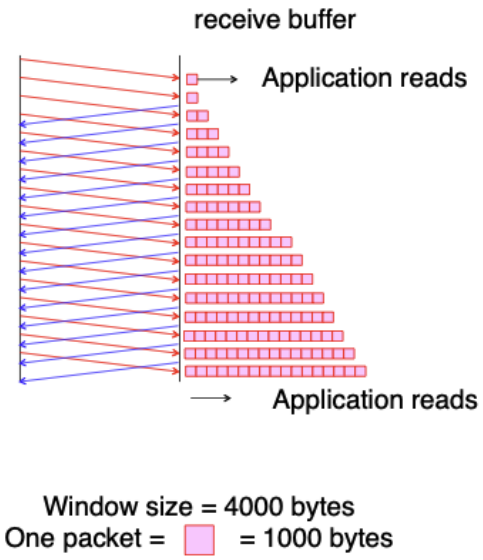
Answer: B.
Packet 4 it outside the window [0:3] and cannot be sent. It has to wait until the loss of packet 0 is repaired, which happens at time t_2

Question follow-up conclusions:

- Sender needs a *retransmission buffer* to hold unack'ed data; its size is equal to the window size.
- Segments are removed from the resequencing/receive buffer when they are finally put in order and application reads them.

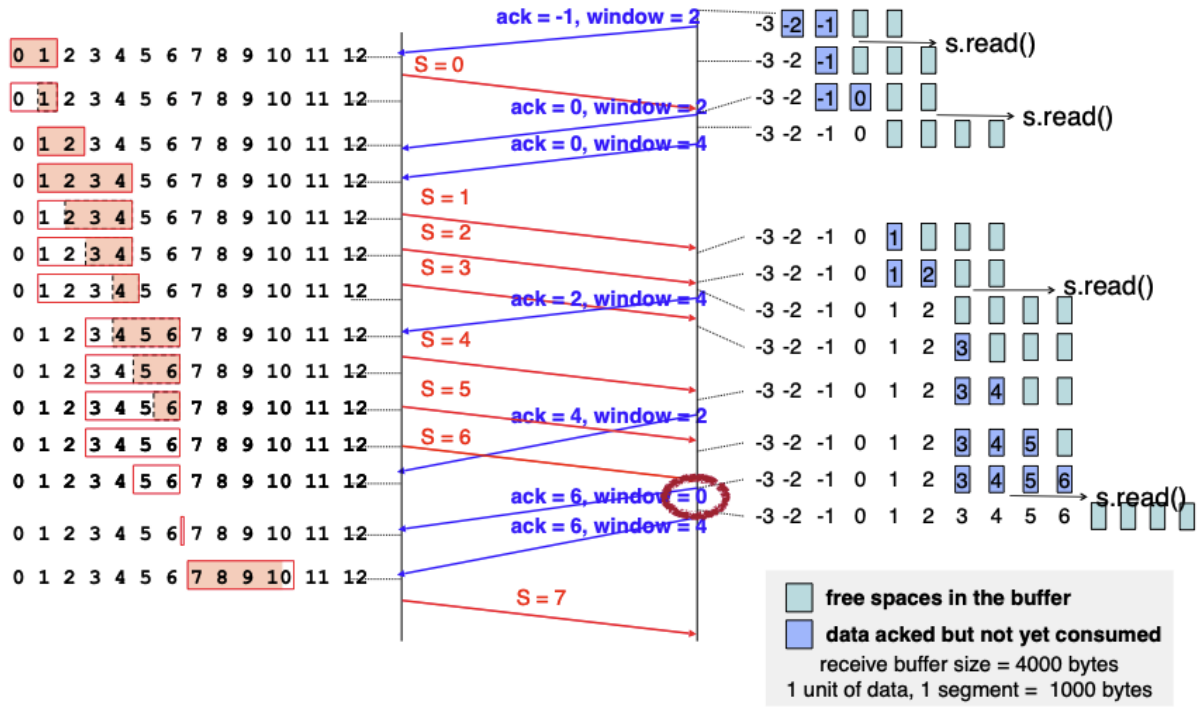
A fixed-size (static) window cannot prevent receive-buffer *overflows*

- Sliding window limits the amount of unack'ed/out-of-order data,
- but in-order data remains in buffer, until applcation reads
- *Slowly reading* receiver app may still trigger buffer overflows



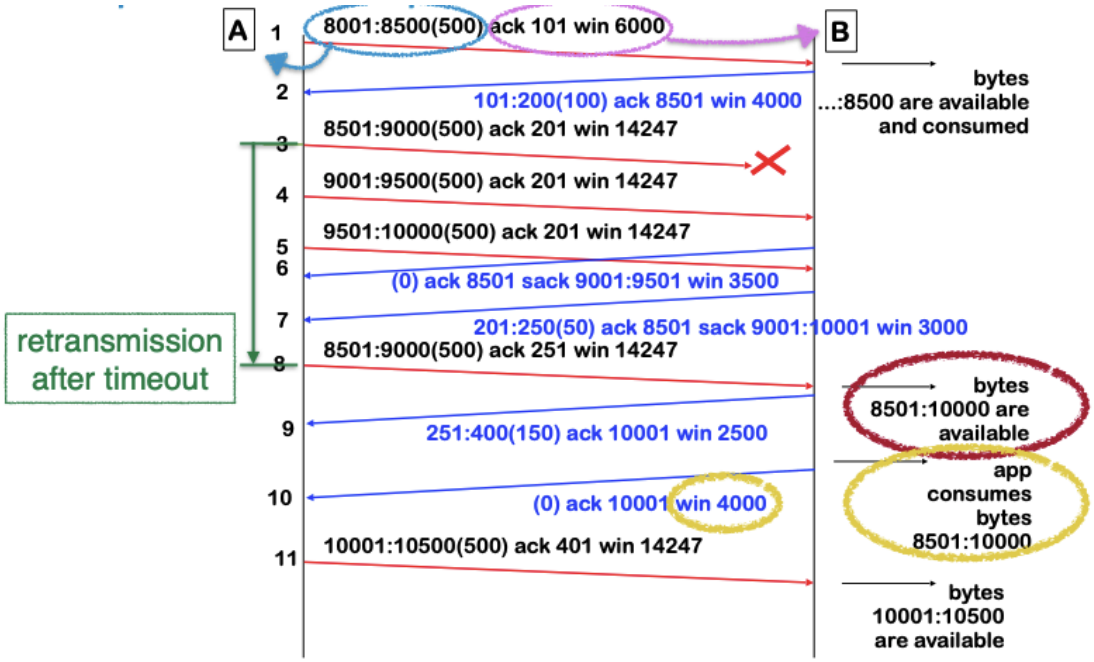
Flow control: an *adaptive* sliding window

- What? adapt the window size to *advertisements* form the other end:
  1. receiver advertizes a window size = available buffer size
     - if no space in buffer, advertized window = 0
  2. sender adapts its sliding window to the advertized value
- → Flow control adapts source's sending rate to the receiver's *consuming speed*
- ≠ **Congestion Control**, which adapts source's sending rate to ***network conditions***



Note: the sender can send from time to time a new segments just to re-trigger an acknowledgement from the receiver's side in case of **the window advertizement packets got lost**.

TCP Basic Operation (Recap)



If there's no loss or reordering, an on a link with capacity c bytes/second, the min window size require for sending at the maximum capacity is...

A. $W_{min} = RTT \times c$
B. $W_{min} = \frac{c}{RTT}$
C. $W_{min} = \frac{RTT}{C}$
D. None of the above
E. I do not know

Answer: A.
If the window size is large enough, the window is never fully used and the sender can send at rate c. This case occurs when the total amount of data in flight, $c \times RTT$, is not larger than $\boldsymbol{W}$, i.e. when $W \geq c \times RTT$ (i.e. window bandwidth-delay product)

Note: A side effect of the streaming service is *head of the line blocking* → if a packet is lost, all data following this packet cannot read by application, until loss is repaired.

For which types of apps may TCP's streaming service be an issue (multiple selection is possible)

A. An app using http/1, where we have several TCP connections, one per object
B. An app using http/2, where we have a single TCP connection per webpage
C. A real-time sensor-data streaming application that sends a new measurement every msec
D. None
E. I do not know

Answer: B and C.
Head-of-line blocking may reduce the quality of user's experience in both cases. In the context of HTTP/2 with a single connection: the entire page download will be delayed if a packet is lost in the transfer of one object of the page.

MSS and segmentation

TCP, *not* the application or IP, chooses how to segment data. Segments should *not* be fragmented at source.

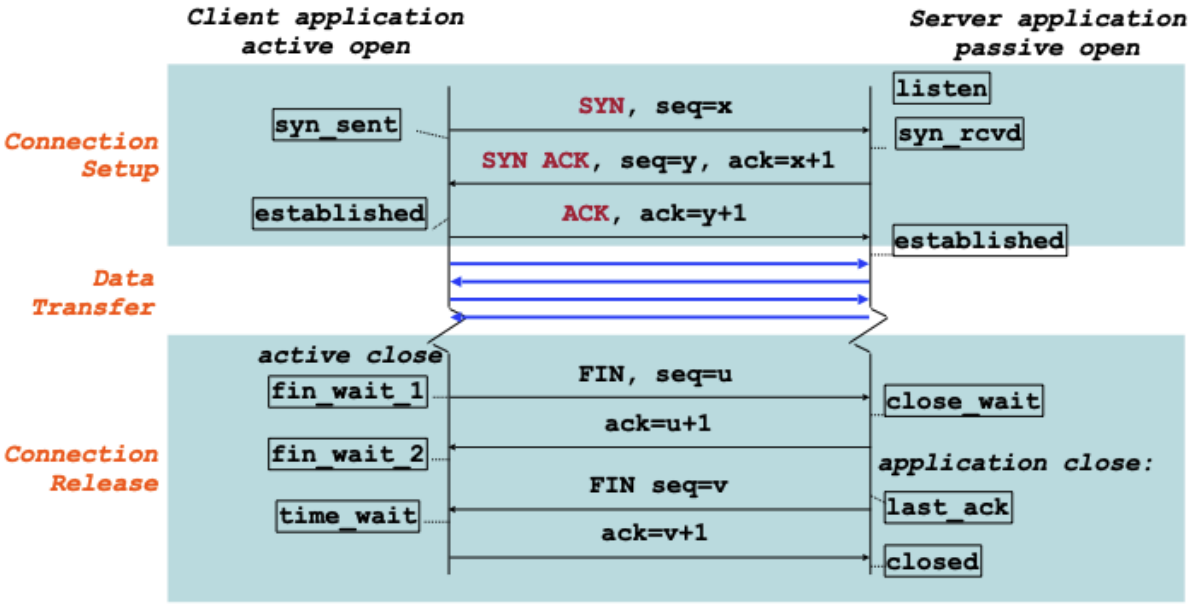
The maximum segment size (MSS) is

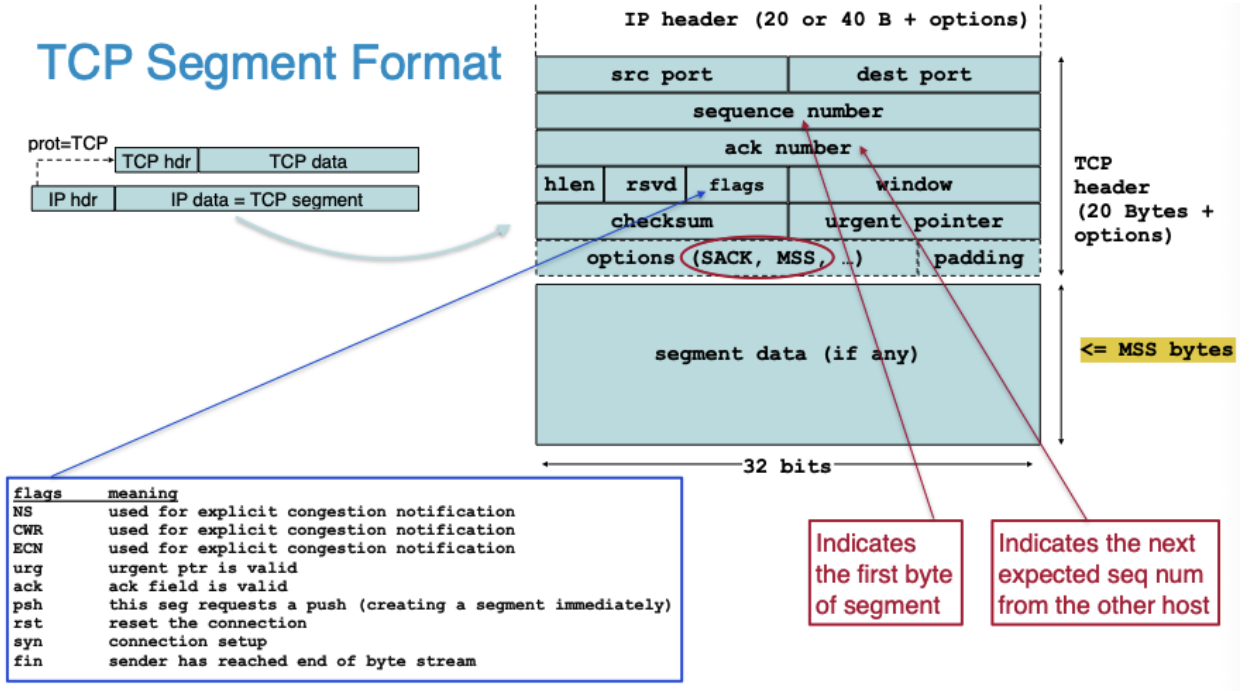
- either a *default* value e.g.:
  - 536 bytes when using IPv4
  - 1220 bytes when using IPv6
- or *negotiated* in Options header field during connection setup: hosts set it to the *smallest value* that both declare.

Modern OSs use TCP Segmentation Offloading (TSO):

- Segmentation is done at the network interface card (NIC) with *hardware* assistance
- Lower CPU load for TCP

3. TCP Connection Phases

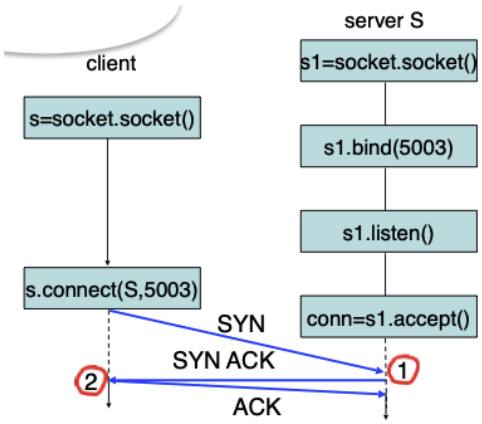




TCP Sockets

More *complicated* than UDP because we need to open/close a connection
A TCP connection requires:

- one side ("server") to *listen* and
- the other side ("client") to *connect*



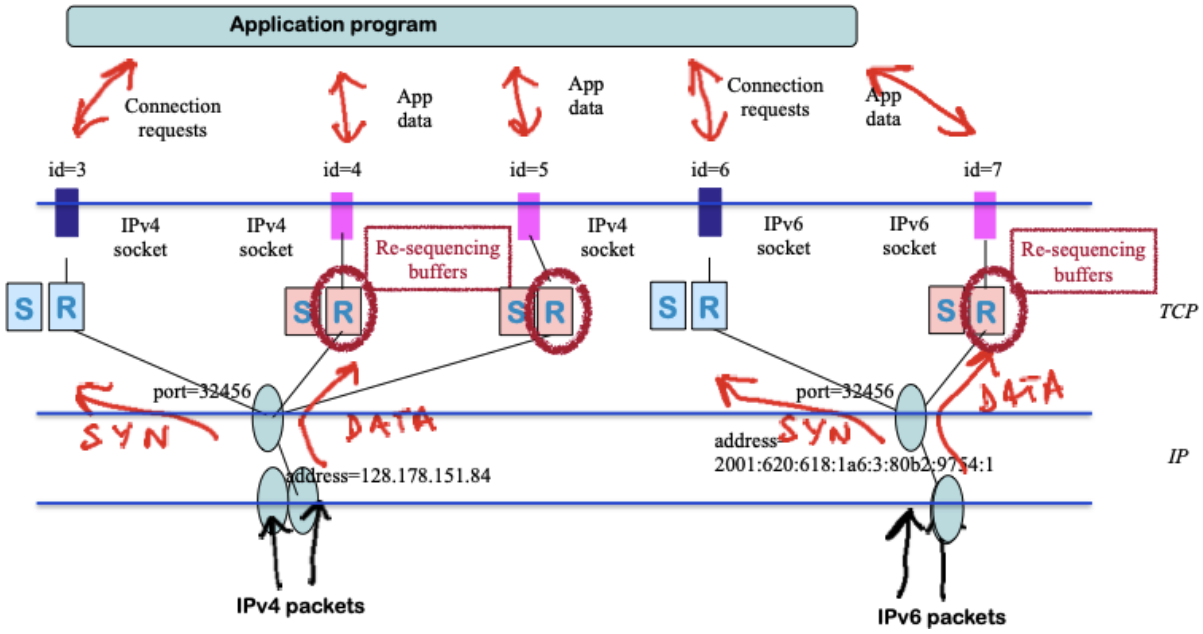
Steps:

1. Server enters *syn-rcvd* state; it stores initial state (client's IP address+port, MSS, etc.), sends SYNACK and waits client's ACK
2. At client, connection is *established*; the socket is used to send/receive data
3. At server, upon reception of the ACK, connection is established and a *new socket (conn)* is created; this will be used to receive/send data

Why a new socket?

- In practice, multiple clients want to connect to the server
- Server creates a new socket per connection
- Uses parallel-execution threads
  - to listen to incoming connection
  - to handle several TCP connections
- A TCP connection is identified by: **{src IP addr, src port, dest IP addr, dest port}**

How the Operating System view TCP socket?



Why both TCP and UDP?

Most applications nowadays use TCP, as this avoids re-inventing error recovery for each app. But some apps may not need TCP-like error recovery (i.e., packet retransmission):

- Voice applications / Sensor data streaming
  Q. Why?
  Delay is important for interactive voice, while packet retransmission may introduce too much delay in some cases.
  Sensor data streaming may send a new packet every few msecs, better to receive latest packet than to repeat a lost one.
- An application that sends just one message, like name resolution (DNS)
  Q. Why? TCP sends several packets of overhead before one single useful data message. Such an application is better served by a Stop and Go protocol at the application layer.
- Multicast (TCP does not support multicast IP addresses)

4. More TCP Bells and Whistles

How to deal with **corner cases**? Suppose that an application writes few bytes in socket; should TCP create a segment or wait for more bytes?

- Nagle's algorithm: send `min{bytes written in one RTT, MSS}`
- How to handle silly window-increase advertisements by 1 byte? The idea is similar to Nagle's, we do something simple

Window field is 16 bits $\Rightarrow$ ma throughput even for RTT=1msec is $524 Mbps; can we do better?$

- negotiate a scaling factor of the window during connection setup (RFC1323)

- How to detect packet loss?
- How to avoid SYN flooding (denial of service attack)?
- How to avoid the delay of the 3-way handshake?

How to detect packet loss?

Packet loss in TCP is detect via times and duplicates ACKs.

The main timers are called retransmission timeouts (RTO), it's set for every packet transmission and when a timer expires (= timeout), this is interpreted as severe loss in the channel. Thus, all timers are reset and all un-acknowledged data is marked as needing retransmission

A duplicate ACK = a TCP segment where the cumulative ACK value repeats a previously received ACK value. This is interpreted as not-so-severe loss in the channel.

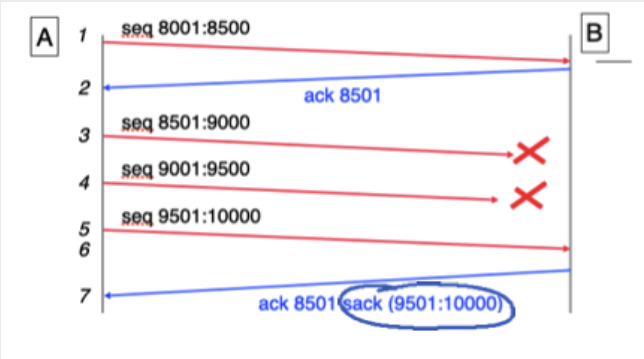
Timers use an estimate of round trip time (RTT)

Why? RTO timer must be set to a value $\geq$ RTT, but not too much

What? RTT estimation gives an upper bound on the round trip time, used to set the RTO

How? it uses a weird formula. RTO increases when there is variability in the RTT. RTO may be significantly larger until it converges back after a sharp discontinuity.

Idea: "Instead of waiting for a timeout, TCP can determine that a packet is lost just after a *single* duplicate ACK is received with a SACK field, and it can quickly retransmit."
Is it a good idea?



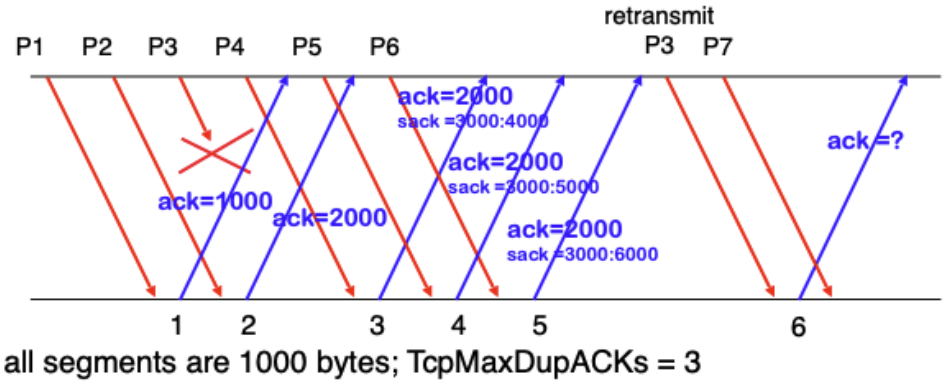
A. Yes because it is likely that there is some missing data
B. No as it may cause superfluous retransmissions (some data could simply be late – out of order)
C. NO because an ACK could be lost
D. I do not know

Answer: There is no "good" answer.
Both A and B can be argued. This is why "Fast Retransmit" was invented.

Fast Retransmit

Principle: when n duplicate ACKs are received, declare a loss

- n = **TcpMaxDupAcks** is set by the Operating System (*default value is 3*)
- Which data is lost is inferred from the SACK blocks



When does Fast Retransmit fail to detect loss via duplicate ACKs?

A. Almost never, it is quasi-optimal
B. It fails to detect the loss extremely rarely, but it may often take a long time to detect.
C. When one of the last segments of an application layer block is lost, fast retransmit does not detect it.
D. It may often fail due to packet re-ordering
E. I don't know

Answer: C (and probably D).

Most losses are detect via Fast Retransmit (duplicate ACKs), but *few go undetected:*

- Losses are not detect, if (forward or reverse) channel is broken
- Isolated/single segments, that are lost, are not detected
- Last/trailing segments (of application data), that are lost, may not be detected
  - *Tail Loss Probe* avoids the latter issue.
- Of course, undetected losses will be eventually detected via RTO timer, but this may take a long(er) time

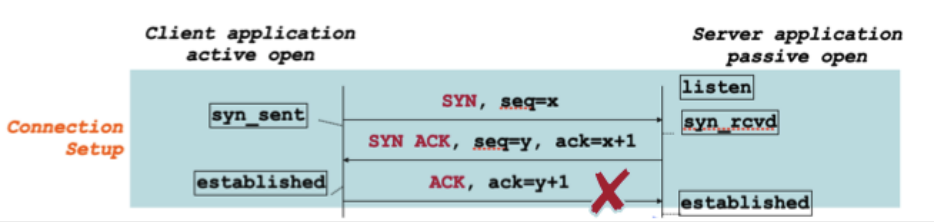
Fast Retransmit may be *inefficient* if packets are just *reordered*

- The sender retransmits packets that weren't lost - they were just reordered
- 2 counter measures:
  - [at network layer]: *per-flow load balancing* avoids sending packets of the same TCP connection over different paths

- [at transport layer]: instead of duplicate ACK, use *RACK (Recent ACK)*, which bases retransmissions on *time*, not sequence numbers:
  - [Alg.:]: Determine that packet p is lost, if:
    - Another out-of-order packet is ack'ed and
    - p has not been ack'ed after time = the latest RTT measurement + a configurable "reordering (time) window"

If ACK (3) is lost during connection setup, server will:

1. retransmit SYN-ACK
2. keep state information until a timeout occurs



A. 1
B. 2
C. 1 and 2
D. None
E. I don't now

Answer: C.
When the server receives a SYN packet, it sends the SYN-ACK and enters a **SYN-received state**, where it keeps all necessary state information about the connection (e.g. IP address and port of the client, the seq number, etc) and does all the usual stuff to detect whether the SYN-ACK is lost. Note that the SYN-ACK does not contain any data in the payload, but it is treated as a normal segment (this is why ACK(3) is y+1 and not y).

Thus, if the SYN-ACK and/or ACK(3) are lost, server will retransmit the SYN-ACK. And if there is still no ACK(3) received after retransmission, the server will retransmit again and again until another timer (indicating that the client is not responding) expires.

In the first TCP implementations, once TCP entered the SYN-received state, it would stay there for several seconds believing that the SYNACK was lost and retransmitting.

But this opened the window to a new type of attack!

Syn flooding attack



- Server receives lots of bogus SYN packets from (spoofed) source addresses
- Keeps state in kernel → runs out of memory → legitimate SYN packets are dropped

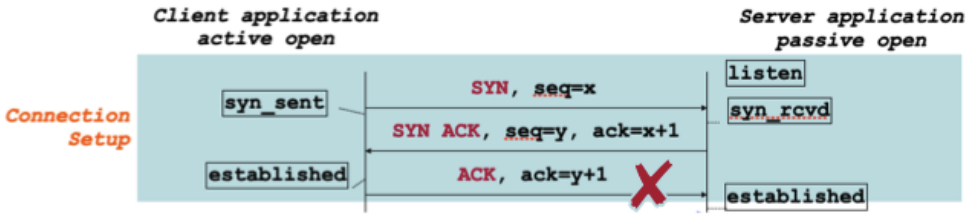
→ Denial of Service!

Solution: SYN cookie encodes state in SEQ of SYN ACK

- Upon receiving a SYN packet, server keeps *no state* and sends a special SEQ number = cookie.
- Legitimate client sends ACK; server verifies the hash and creates a socket with MSS from cookie
- Bogus client sends no ACK; server has lost some time to compute hash, but reserved no memory

→ SYN flooding is mitigated

What if server uses SYN cookies and ACK (3) is lost?



- If client app sends data with ACK (3), next segment servers as an ACK
  - server sees it, validates the cookie, and establish state
- If client apps sends no data with ACK (3), we risk a *deadlock*
  - server receives no ACK → creates no socket
  - ACK (3) is never acknowledged, hence never retransmitted
  - → client hangs on established state
  - → such apps (e.g., SSH or mySQL) implement a timeout at the application layer to avoid this

With SYN cookies, the response time of SYN-ACK is...

A. Larger than without SYN cookies
B. Smaller than without SYN cookies
C. The same
D. I don't know

Answer: A. Cryptographic calculations take time... Hence, damage from SYN flooding is reduced to *increased computation time*, but memory exhaustion is avoided